

Simple cyber security system that demonstrates **secure authentication principles**

Github link -> <https://github.com/tetrax-prog/cybersec-cat2.git>

Taking user data from frontend. It takes 3 values username, password, and confirm password

```
//register logic
if(isset($_POST["register"])){
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        $username = $_POST['username'];
        $pass = $_POST['password'];
        $confirm_pass = $_POST['confirm-password'];
```

Validate password strength. It checks:

- Minimum length (≥8 characters)
- Use of uppercase letters
- Use of lowercase letters
- Use of numbers

```
//password and confirm password check
if($pass != $confirm_pass){
    redirect(url: "register.php", status: "Password and confirm password are not same",messageType: "danger");
}
// 1. Length Check
if (strlen(string: $pass) < 8) {
    redirect(url: "register.php", status: "Password must be at least 8 characters long.",messageType: "danger");
}
// 2. Uppercase Check
if (!preg_match(pattern: '/[A-Z]/', subject: $pass)) {
    redirect(url: "register.php", status: "Password must include at least one uppercase letter.",messageType: "danger");
}
// 3. Lowercase Check
if (!preg_match(pattern: '/[a-z]/', subject: $pass)) {
    redirect(url: "register.php", status: "Password must include at least one lowercase letter.",messageType: "danger");
}
// 4. Number Check
if (!preg_match(pattern: '/[0-9]/', subject: $pass)) {
    redirect(url: "register.php", status: "Password must include at least one number.",messageType: "danger");
}
```

Hashing of password is done using PASSWORD_DEFAULT which uses a BCrypt hashing algorithm

```
$hashed_password = password_hash(password: $pass, algo: PASSWORD_DEFAULT);
```

Adding the registered user to the database. Database used is MariaDB.

```
//Insert into Database using Prepared Statements
try {
    // 2. Prepare the SQL statement
    $sql = "INSERT INTO users (username, password_hash) VALUES (:username, :password)";
    $stmt = $pdo->prepare(query: $sql);

    // 3. Execute with the data
    $stmt->execute(params: [
        'username' => $username,
        'password' => $hashed_password
    ]);

    redirect(url: "login.php", status: "Registration successfull.",messageType: "success");
```

Below is the authentication code. The authentication code hashes the password entered and checks it against the stored hash found in the database. It also limits the login trials to three. Once login trials are exceeded account is locked

```
//login logic
if(isset($_POST["login"])){
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        $username = $_POST['username'];
        $password = $_POST['password'];

        // 1. Fetch user data
        $stmt = $pdo->prepare(query: "SELECT id, password_hash, failed_attempts FROM users WHERE username = :username");
        $stmt->execute(params: ['username' => $username]);
        $user = $stmt->fetch();

        if ($user) {
            // 2. Check if the account is locked
            if ($user['failed_attempts'] >= 3) {
                redirect(url: "login.php", status: "Account locked. Too many failed attempts.", messageType: "danger");
            }

            // 3. Verify password
            if (password_verify(password: $password, hash: $user['password_hash'])) {
                // SUCCESS: Reset failed attempts to 0
                $reset = $pdo->prepare(query: "UPDATE users SET failed_attempts = 0 WHERE id = :id");
                $reset->execute(params: ['id' => $user['id']]);

                $_SESSION['user_id'] = $user['id'];
                echo "Login successful! Welcome.";
            } else {
                // FAILURE: Increment failed attempts
                $new_attempts = $user['failed_attempts'] + 1;
                $update = $pdo->prepare(query: "UPDATE users SET failed_attempts = :attempts WHERE id = :id");
                $update->execute(params: ['attempts' => $new_attempts, 'id' => $user['id']]);

                $remaining = 3 - $new_attempts;
                echo "Invalid username or password. Attempts remaining: " . ($remaining > 0 ? $remaining : 0);
                redirect(url: "login.php", status: "Invalid password. Attempts remaining: " . ($remaining > 0 ? $remaining : 0), messageType: "danger");
            }
        } else {
            redirect(url: "login.php", status: "Invalid username ", messageType: "danger");
        }
    }
}
```

User information is stored in MariaDB relational database.

Extra options

		id	username	password_hash	failed_attempts	last_attempt	created_at		
☐	 Edit	 Copy	 Delete	2	Tetrax	\$2y\$10\$etuMORMXI5gzGPO/Z6ICZuilhENjebLJoBbieJb9Zau...	0	NULL	2026-02-17 10:40:15
☐	 Edit	 Copy	 Delete	4	Noob	\$2y\$10\$XXatNtckQEgMOMYkmy/PQu1qUyy86byurw4IKJOZfbz...	0	NULL	2026-02-17 11:24:25
☐	 Edit	 Copy	 Delete	5	Saibot	\$2y\$10\$19ywZTzba..dl52cTib8YuMZ5/G.k1xKaRz0qUvrOVq...	0	NULL	2026-02-17 11:25:02
 Check all With selected:  Edit  Copy  Delete  Export									

User interface

CREATE AN ACCOUNT

Your username

Password

Repeat your password

☐ I agree all statements in [Terms of service](#)

Register

Password must include at least one uppercase letter.

Your username

Password

Repeat your password

Password must be at least 8 characters long.

Your username

Password

Repeat your password

LOGIN

Your username

Password

Login

Dont have an account? [Register here](#)

LOGIN

Invalid password. Attempts remaining: 2

Your username

Password

Login

LOGIN

Account locked. Too many failed attempts.

Your username

Password

Login